

Module - 2(Filled Area Primitives and transformations)

Filled Area Primitives- Scan line polygon filling, Boundary filling and flood filling.

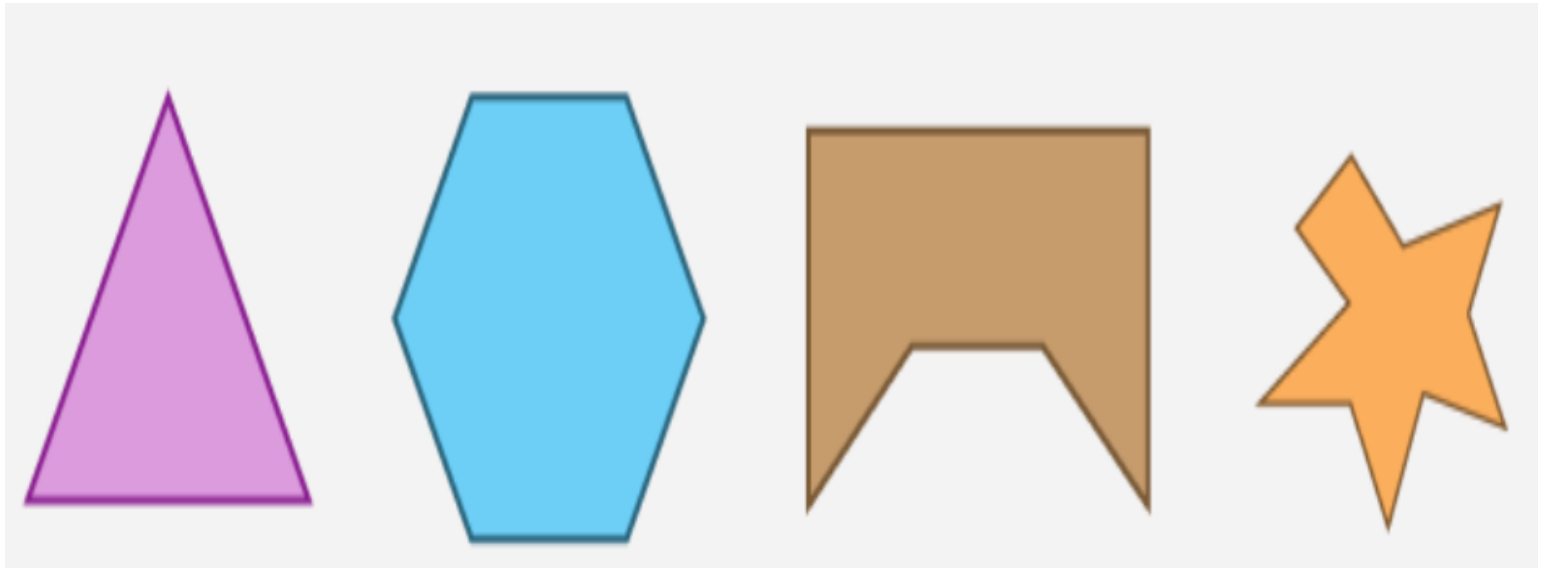
Two-dimensional transformations-Translation, Rotation, Scaling, Reflection and Shearing, Composite transformations, Matrix representations and homogeneous coordinates.

Basic 3D transformations.

Part - 1

POLYGON FILL ALGORITHMS

In geometry, a **polygon** can be defined as a flat or plane, two-dimensional closed shape bounded with straight sides. It does not have curved sides.



Filled Area Primitives

- One standard output primitive in general graphics packages is a solid-color or patterned polygon area.
- Other kinds of area primitives are sometimes available, but polygons are easier to process since they have linear boundaries
- Two basic approaches to area filling on raster systems.
 - determine the overlap intervals for scan lines that cross the area.
 - to start from a given interior position and paint outward from this point until we encounter the specified boundary conditions.
- The scan-line approach is typically used in general graphics packages to fill polygons, circles, ellipses, and other simple curves.
- Fill methods starting from an interior point are useful with more complex boundaries and in interactive painting systems.

Inside-Outside Tests

Area filling algorithms often need to identify interior regions of objects.

- To determine a point lies inside a polygon or not, in computer graphics, we have two methods :

1.Odd Even Rule (Odd Parity Rule)

2.Non Zero winding number Rule

1.Odd Even Rule (Odd Parity Rule)

1. Construct a line segment from point to be examined to point outside of a polygon.
2. Count the number of intersections of line segment with polygon boundaries.
3. If Odd number of intersection, then Point lies inside of Polygon.
4. Else, Point lies outside of polygon.

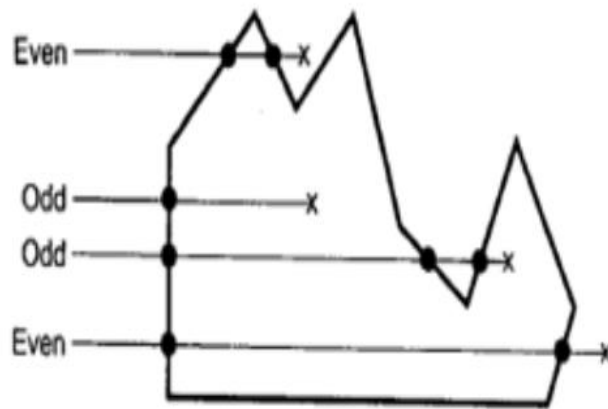


Fig (a)

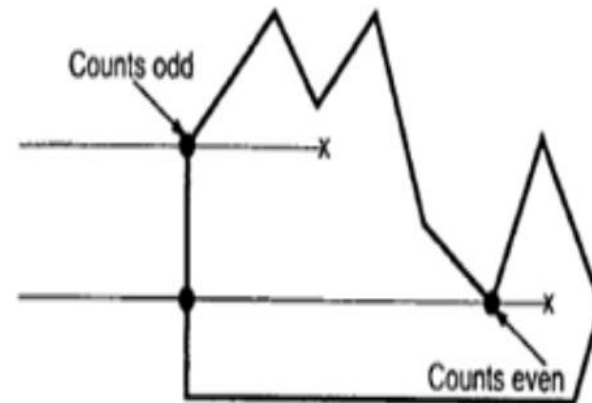
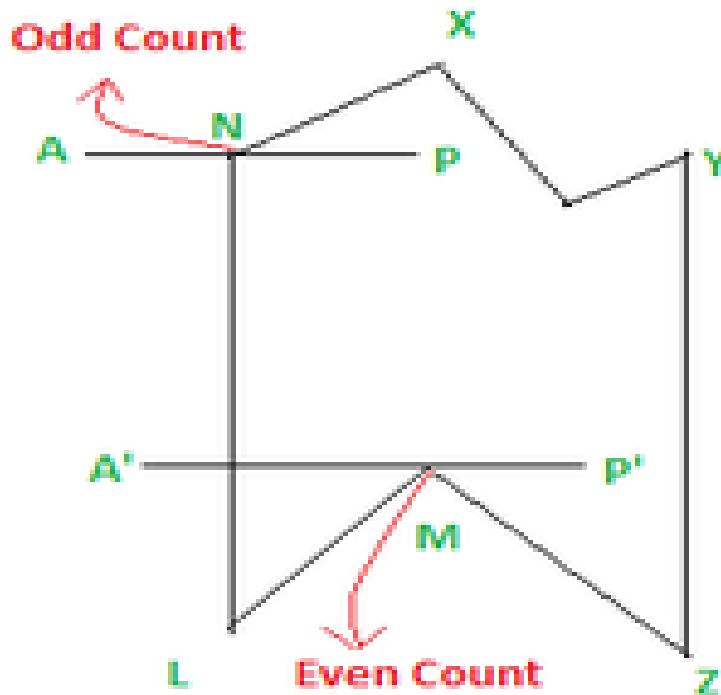


Fig (b)

Scan line that cut a vertex is a special case ;

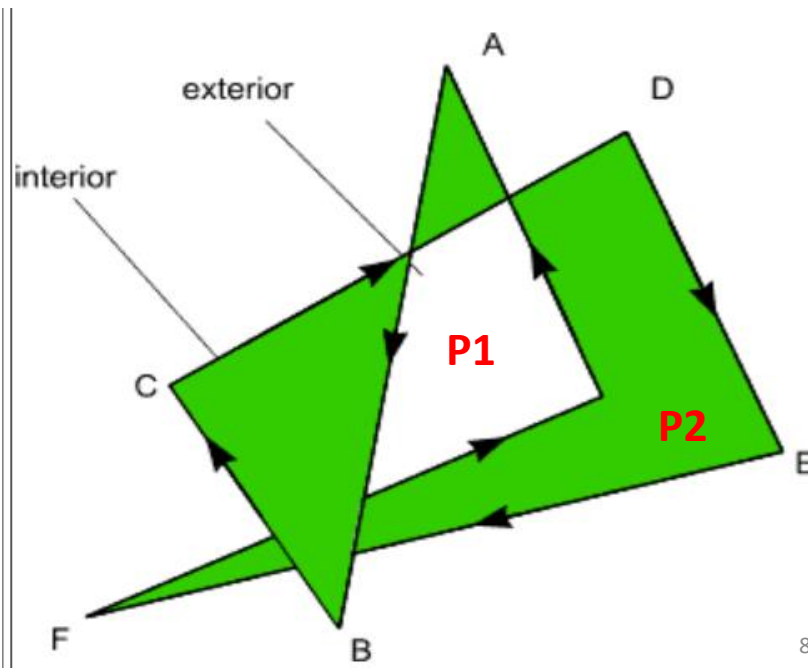
Case 1 : Consider edges intersect at a vertex, if their other endpoints are on the same side of the scan line, count number of intersections as 2(or even).

Case 2 : Consider edges intersect at a vertex, if their other endpoints are on the opposite sides of the scan line ,count number of intersections as 1(or odd).



2.Non Zero winding number Rule

- 1) Initial value of winding number=0.
- 2) Then draw a line from point (p-point to test) to outside the polygon which does not pass through any vertex.
- 3) Add 1 to the winding number every time we intersect a polygon edge that crosses the line from right to left and subtract 1 every time we intersect an edge that crosses from left to right.
- 4) The interior points are that have a non zero value for the winding number .
- 5) Exterior points are those whose value of the winding number is zero .



**LEFT TO RIGHT:-1
(CLOCKWISE)**
**RIGHT TO LEFT: +1
(COUNTER-
CLOCKWISE)**
INITIAL VALUE = 0

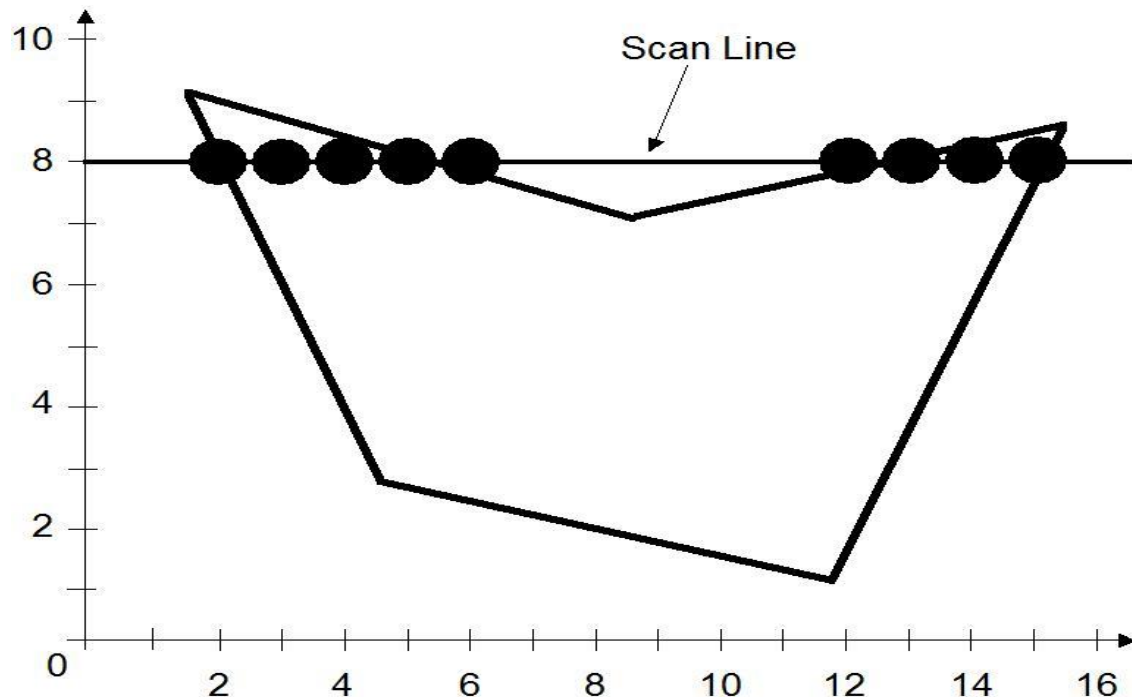
1. Scan line Polygon fill Algorithm

- For each scan line crossing a polygon, the area-fill algorithm locates the intersection points of the scan line with the polygon edges.
- These intersection points are then sorted from left to right, and the corresponding frame-buffer positions between each intersection pair are set to the specified fill colour.

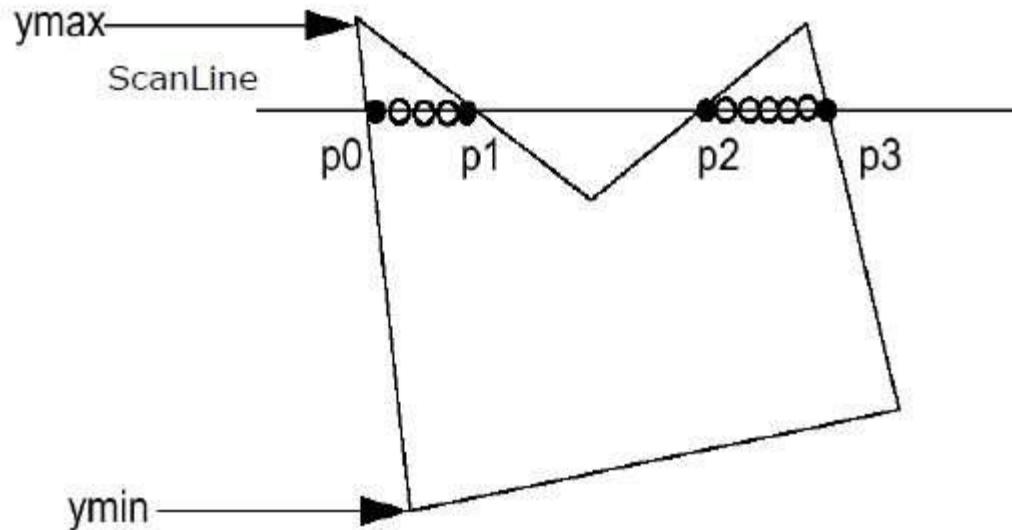
The Scan-Line Polygon Fill Algorithm

The scan-line polygon-filling algorithm involves

- the **horizontal scanning** of the polygon from its **lowermost** to its **topmost** vertex,
- identifying which **edges intersect the scan-line**,
- and finally **drawing the interior horizontal lines** with the specified **fill color** process.



Scan-Line Polygon Fill Algorithm



This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections. The following steps depict how this algorithm works.

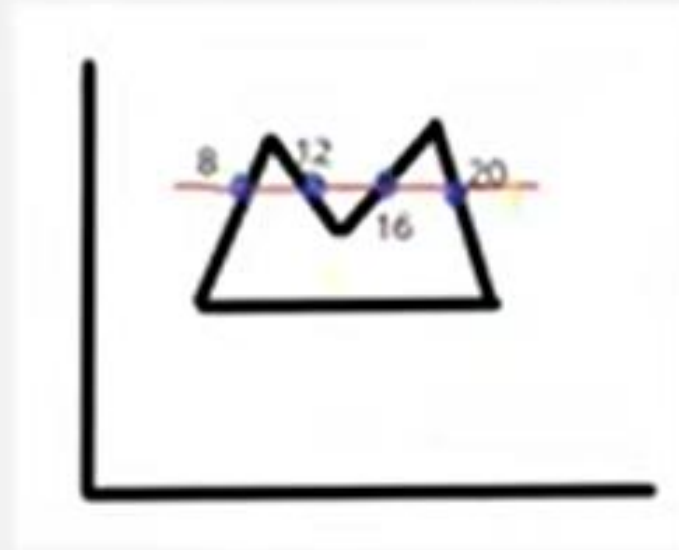
Step 1 – Find out the Ymin and Ymax from the given polygon.

Step 2 – ScanLine intersects with each edge of the polygon from Ymin to Ymax. Name each intersection point of the polygon. As per the figure shown above, they are named as p0, p1, p2, p3.

Step 3 – Sort the intersection point in the increasing order of X coordinate i.e. (p0,p1),(p1, p2) and (p2,p3).

Step 4 – Fill all those pair of coordinates that are inside polygons and ignore the alternate pairs.

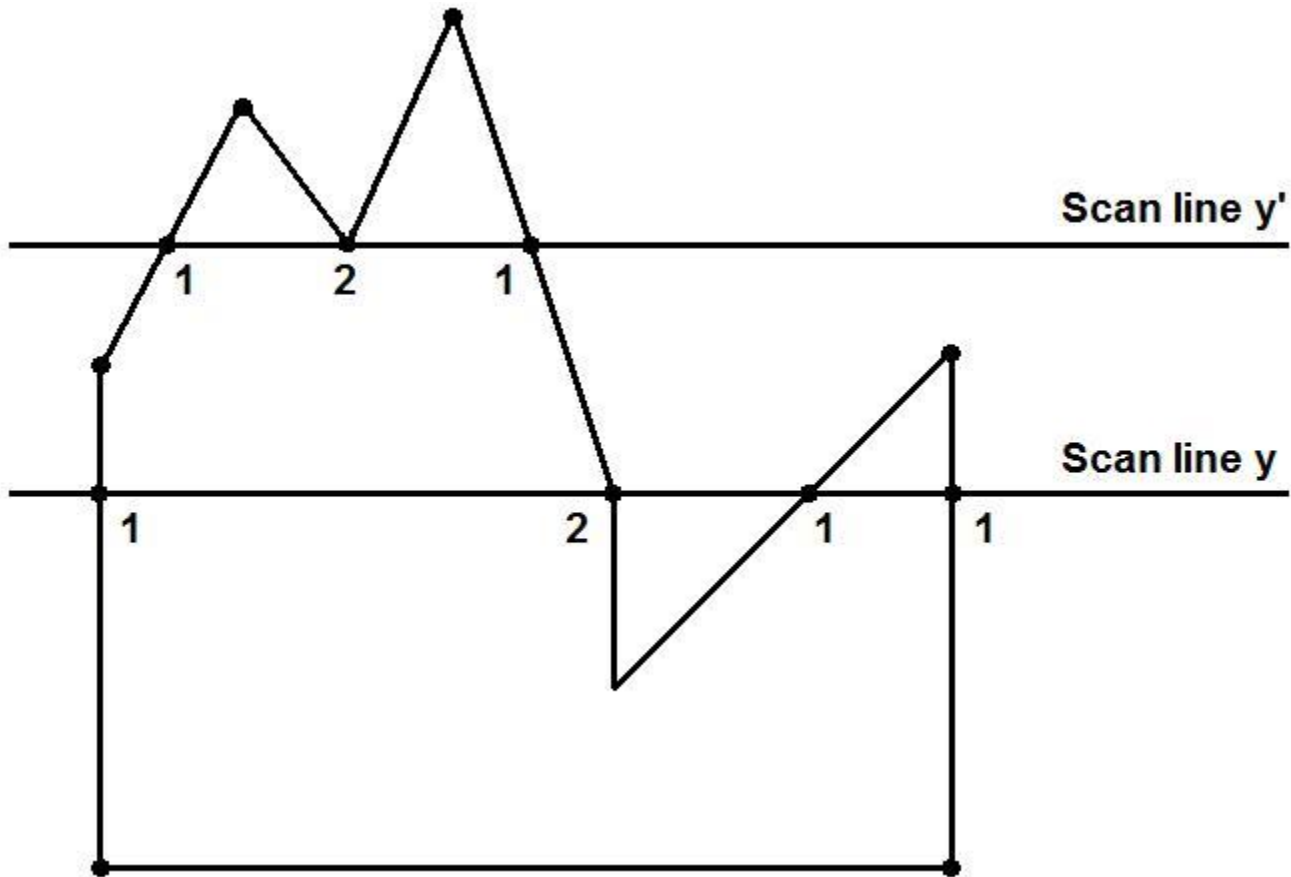
Simple example



- List of intersection points $\rightarrow 8, 12, 16, 20$
- Sorted order will be $\rightarrow 8, 12, 16, 20$
- Make pairs of intersection $\rightarrow (8, 12), (16, 20)$
- Fill in all the pixels with the given colour inside the pixel

The Scan-Line Polygon Fill Algorithm

Dealing with vertices



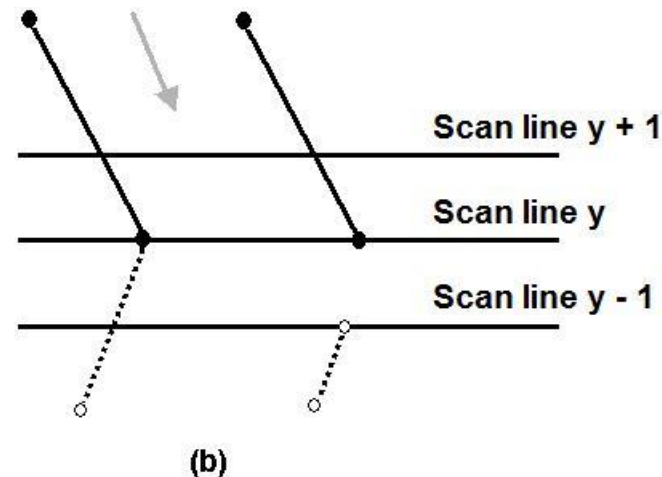
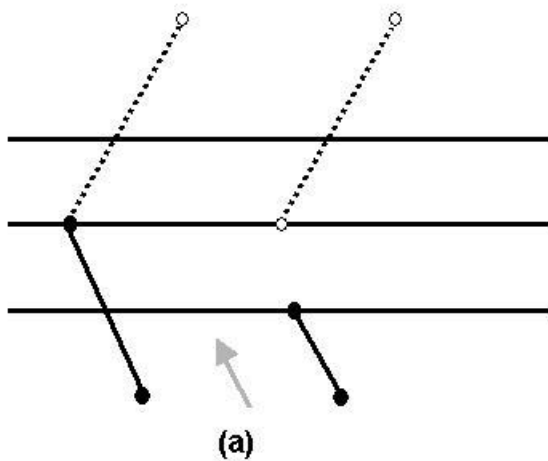
Scan-Line Polygon Fill

- Some scan-line intersections at polygon vertices require special handling
- Scan line passing through a vertex intersects two polygon edges at that position, adding two points to the list of intersections for the scan line y' correctly identify the interior pixel spans.
- But with scan line y , we need to do some additional processing to determine the correct interior points
- For scan line y , the two intersecting edges sharing a vertex are on opposite sides of the scan line. But for scan line y' , the two intersecting edges are both above the scan line.
- Thus, the vertices that require additional processing are those that have connecting edges on opposite sides of the scan line.
- We can identify these vertices by tracing around the polygon boundary either in clockwise or counterclockwise order and observing the relative changes in vertex y coordinates as we move from one edge to the next.

The Scan-Line Polygon Fill Algorithm

Dealing with vertices

- When the endpoint **y** coordinates of the two edges are **increasing**, the **y** value of the upper endpoint for the **current edge** is decreased by one (a)
- When the endpoint **y** values are **decreasing**, the **y** value of the **next edge** is decreased by one (b)



- Calculations performed in scan-conversion and other graphics algorithms typically take advantage of various coherence properties of a scene that is to be displayed
- Coherence methods often involve incremental calculations applied along a single scan line or between successive scan lines
- In determining edge intersections, we can set up incremental coordinate calculations along any edge by exploiting the fact that the slope of the edge is constant from one scan line to the next.

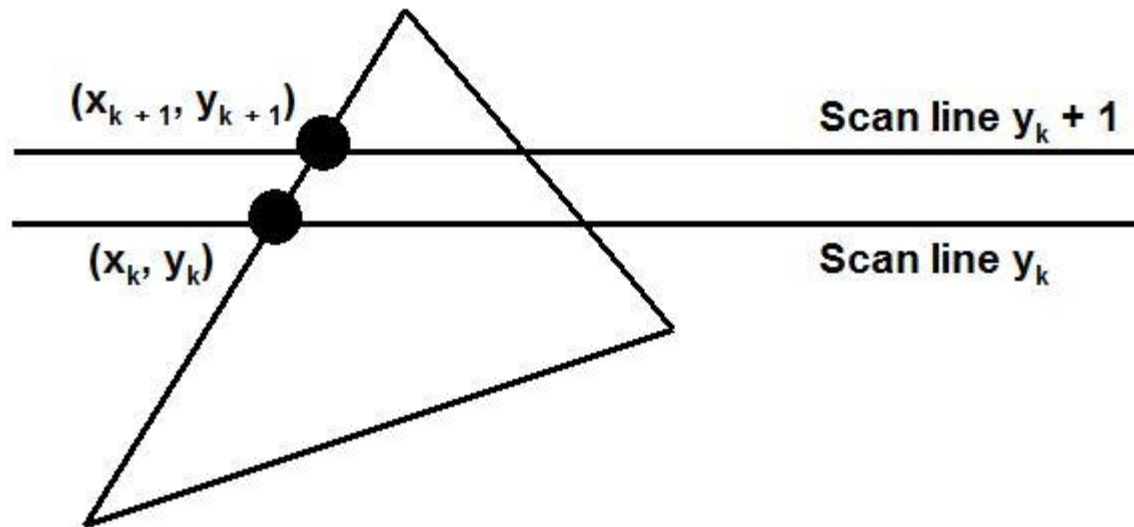
The Scan-Line Polygon Fill Algorithm

Determining Edge Intersections

$$m = (y_{k+1} - y_k) / (x_{k+1} - x_k)$$

$$y_{k+1} - y_k = 1$$

$$x_{k+1} = x_k + 1/m$$



The Scan-Line Polygon Fill Algorithm

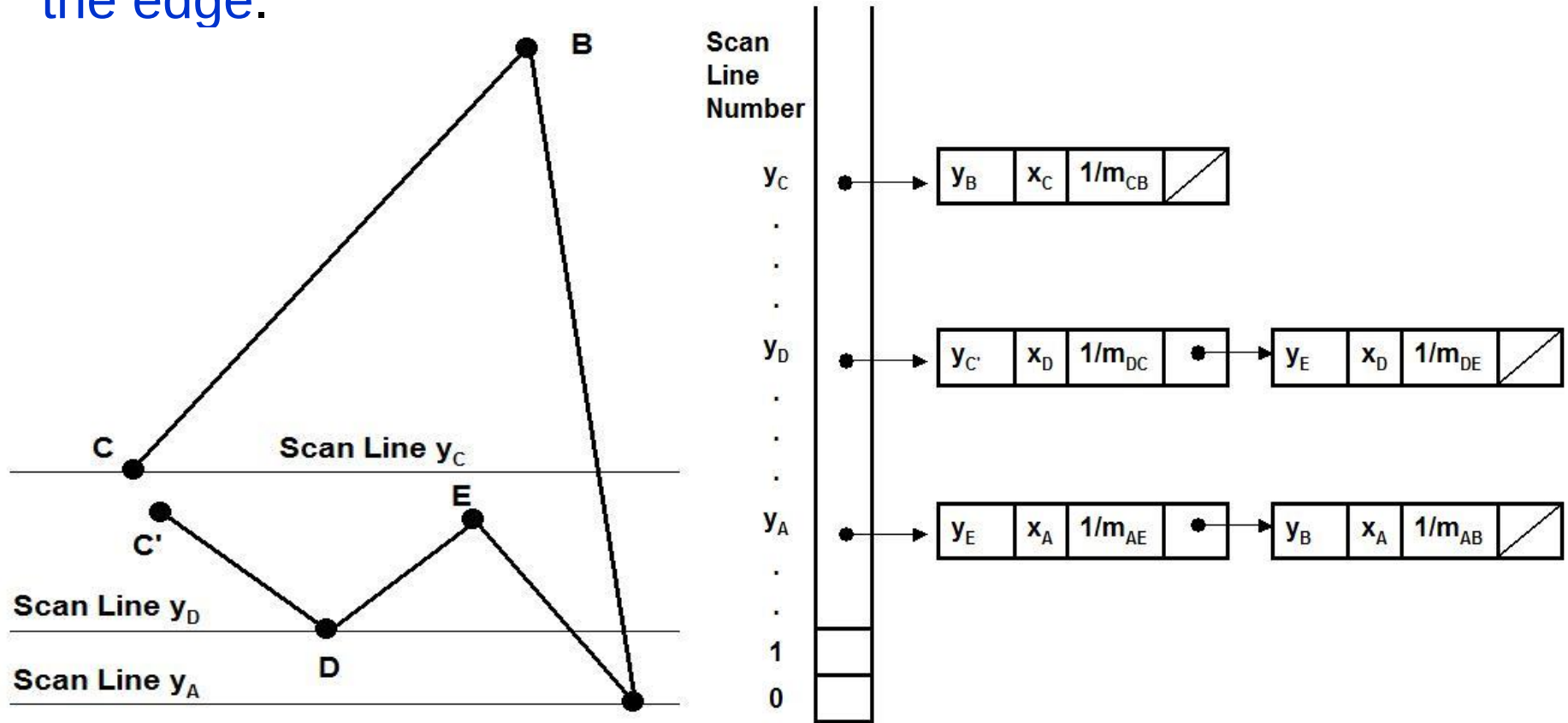
- To efficiently perform polygon filling, first store **polygon boundary in sorted edge table**.
- Proceeding around the edges in either a **clockwise or a counterclockwise order**, we can use a **bucket sort** to store the edges, **sorted on the smallest y value of each edge**, in the correct scan line positions
- As the edges are processed, we can **shorten certain edges to resolve the vertex intersection question**
- **Each entry in the table** for a particular scan line contains the **maximum y value for that edge, the x-intercept value (at the lower vertex) for the edge, and the inverse slope of the edge**.
- For each scan line, the **edges are in sorted order from left to right**.

The Scan-Line Polygon Fill Algorithm

- Next, we process the scan lines from the bottom of the polygon to its top, producing an active edge list for each scan line crossing the polygon boundaries.
- The active edge list for a scan line contains all edges crossed by that scan line, with iterative coherence calculations used to obtain the edge intersections

The Scan-Line Polygon Fill Algorithm

- Each **entry** in the table for a particular scan line contains the **maximum y** value for that edge, the **x-intercept** value (at the lower vertex) for the edge, and the **inverse slope** of the edge.



A polygon and its sorted edge table, with edge DC shortened by one unit in the y direction.

Boundary Fill Algorithm

- Another approach to area filling is to **start at a point inside a region and paint the interior outward toward the boundary.**
- If the boundary is specified in a single color, the fill algorithm proceeds outward pixel by pixel until the boundary color is encountered.
- This is called the boundary-fill algorithm, is particularly useful in interactive painting packages, where interior points are easily selected.

A boundary fill procedure:

1. Accepts as input the coordinates of an interior point (x, y) , a fill color, and a boundary color.
2. Starting from (x, y) , the procedure tests neighboring positions to determine whether they are of the boundary color.
3. If not, they are painted with the fill color, and their neighbors are tested.
4. This process continues until all pixels up to the boundary color for the area have tested

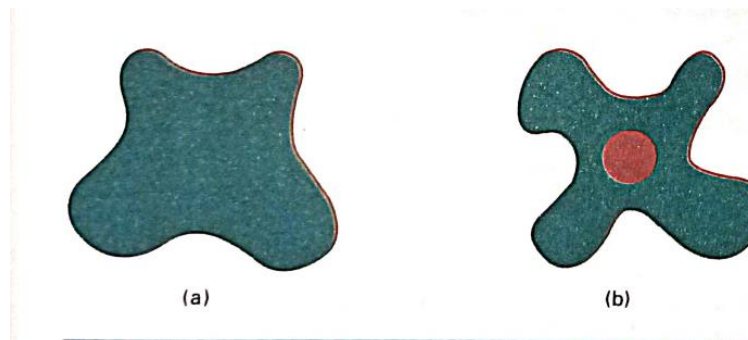
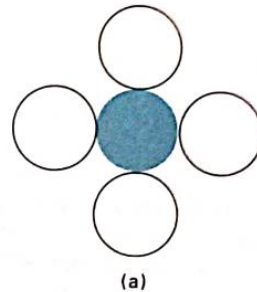


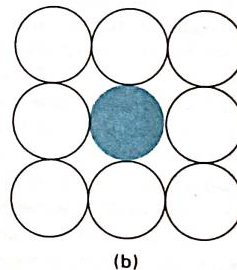
Figure 3-42

Example color boundaries for a boundary-fill procedure.

- Two methods are used for proceeding to neighboring pixels from the current test position.
 - 4 Connected: Here 4 neighboring points are tested. These are the pixel positions that are right, left, above and below the current pixel. Areas filled by this method are called 4-connected.



- 8 Connected: The second method is used to fill more complex figures. Here the set of neighboring positions to be tested includes the four diagonal pixels. Fill methods using this approach are called 8-connected.



- An 8-connected boundary-fill algorithm would correctly fill the interior of the area, but a 4-connected boundary-fill algorithm produces the partial fill .

The following procedure illustrates a recursive method for filling a 4-connected area with an intensity specified in parameter fill up to a boundary colour specified with parameter boundary.

```
void boundaryFill4 (int x, int y, int fill, int boundary)
{
    int current;
    current = getpixel (x, y);
    if ((current != boundary) && (current != fill))
    {
        setcolour (fill);
        setpixel (x, y);
        boundaryfill4 (x+1, y, fill, boundary);
        boundaryFill4 (x-1, y, fill, boundary);
        boundaryFill4 (x, y+1, fill, boundary);
        boundaryFill4 (x, y-1, fill, boundary);
    }
}
```


Flood-Fill Algorithm

- To fill in (or recolour) an area that is not defined within a single colour boundary, paint such areas by replacing a specified interior colour instead of searching for a boundary colour value. This approach is called a **flood-fill algorithm**.
- Start from a specified interior point (x, y) and reassign all pixel values that are currently set to a given interior colour with the desired fill colour.
- If the area that wants to paint has more than one interior colour, first reassign pixel values so that all interior points have the same colour.
- Using either a 4 connected or 8-connected approach, then step through pixel positions until all interior points have been repainted.
- The following procedure flood fills a connected region recursively, starting from the input position.

```
void floodFill4 (int x, int y, int fillColor, int oldColor)
{
    if (getPixel (x, y) == oldColor) {
        setColor (fillColor);
        setPixel (x, y);
        floodFill4 (x+1, y, fillColor, oldColor);
        floodFill4 (x-1, y, fillColor, oldColor);
        floodFill4 (x, y+1, fillColor, oldColor);
        floodFill4 (x, y-1, fillColor, oldColor);
    }
}
```



Figure 3-46

An area defined within
multiple color boundaries.

Flood-fill Algorithm

It can process the image containing more than one boundary colours.

Flood-fill algorithm is comparatively slower than the Boundary-fill algorithm.

In Flood-fill algorithm a random colour can be used to paint the interior portion then the old one is replaced with a new one.

It requires huge amount of memory.

Flood-fill algorithms are simple and efficient.

Boundary-fill Algorithm

It can only process the image containing single boundary colour.

Boundary-fill algorithm is faster than the Flood-fill algorithm.

In Boundary-fill algorithm Interior points are painted by continuously searching for the boundary colour.

Memory consumption is relatively low in Boundary-fill algorithm.

The complexity of Boundary-fill algorithm is high.